Unit 2:
Data Organizational Structures

Categories of Data

What Is Data?

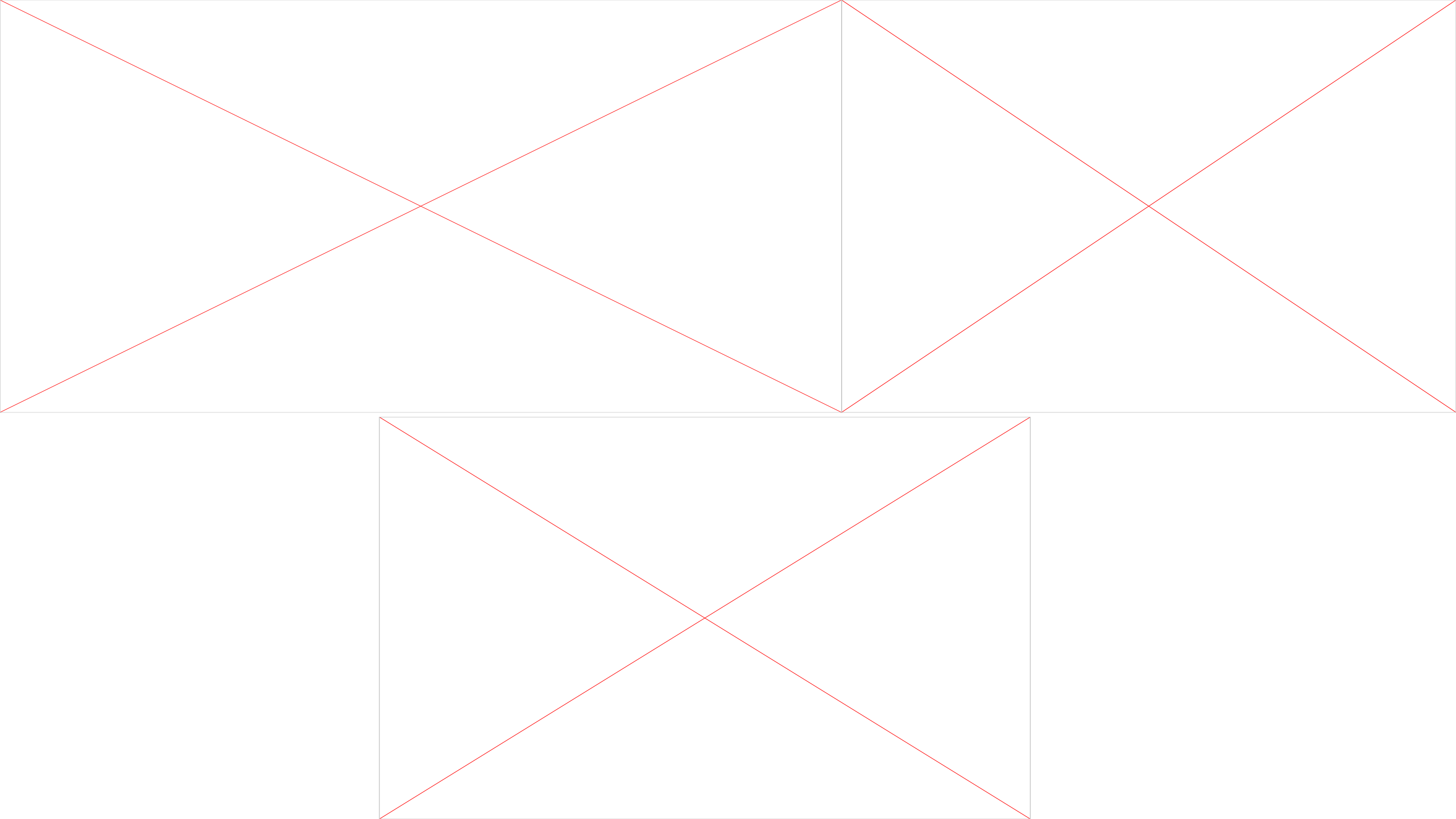
Data is broadly categorized based on:

- **Structure**
- **Organization**
- **How machines understand it**

1. Structured Data

Structured Data is data that:

- Is organized in **rows and columns**
- Follows a **fixed schema**
- Is easy to store, search, and analyze



Examples

- Student marks table
- Employee salary records
- Bank transaction tables
- CSV / Excel files
- SQL databases

Roll No	Name	Marks
1	Ravi	85
2	Anita	90

```
students = [
    [1, "Ravi", 85],
    [2, "Anita", 90]
]
```

2.Unstructured Data

Unstructured Data has:

- No fixed format
- No predefined schema
- High volume, high complexity

Examples

- Text documents
- Emails
- Images
- Audio files
- Videos
- Social Media Post

3.Categorical Data

Categorical Data represents:

- **Labels or categories**
- Qualitative values
- Finite set of options

Common Examples

- **Gender:** Male, Female, Other
- **Blood Group:** A, B, AB, O
- **Color:** Red, Blue, Green
- **Country:** India, USA, Japan
- **Department:** HR, IT, Finance

Types of Categorical Data

a) Nominal (No order)

- Gender: Male, Female
- Blood group: A, B, AB, O
- Color: Red, Blue

b) Ordinal (Order matters)

- Ratings: Poor, Average, Good
- Grades: A, B, C

Real-Life Examples

- Survey responses
- Feedback forms
- Product categories

4.Time Series Data

Time Series Data is:

- Data collected **over time**
- Time order is **mandatory**
- Used for trend and forecasting

Examples

- Daily temperature readings
- Monthly sales
- Stock prices
- Heart rate over time

Comparison Summary

Data Type	Structure	Example
Structured	Fixed	Tables, CSV
Unstructured	No format	Text, images
Categorical	Labels	Gender, Grade
Time Series	Time-based	Weather, Stocks

Topic: List – Creating Lists & List Operations

Learning Objectives

- Create lists using different methods
- Access and modify list elements
- Perform common list operations
- Apply list operations in data-oriented examples

- A **list stores multiple values**
- Order matters
- Can store **heterogeneous data**

```
numbers = [1, 2, 3, 4, 5]
names = ["Ram", "Shyam", "Geeta"]
mixed = [1, "Python", 3.14, True]
```

Operations on list:

1. Create/Insert elements
2. Access element
3. Slicing
4. Modifying List Elements

Creating Lists

1 Creating an Empty List

```
lst = []

data = list()
```

✔️ Use-case: dynamic data collection

2 Creating a List with Values

```
numbers = [1, 2, 3, 4, 5]

names = ["Ram", "Shyam", "Geeta"]

mixed = [1, "Python", 3.14, True]
```

3.Creating List Using `list()` Constructor

```
chars = list("DATA")
```

Strings are iterable

Each character becomes an element

Result:

```
['D', 'A', 'T', 'A']
```

4.Creating List Using `range()`

```
nums = list(range(1, 6))
```

Result:

```
[1, 2, 3, 4, 5]
```

Accessing List Elements

1 Indexing

```
numbers = [10, 20, 30, 40]

print(numbers[0])    # 10

print(numbers[2])    # 30
```

2 Negative Indexing

```
print(numbers[-1])   # 40

print(numbers[-2])   # 30
```

Slicing

```
numbers = [10, 20, 30, 40]
print(numbers[1:3])    # [20, 30]
print(numbers[:2])     # [10, 20]
print(numbers[2:])     # [30, 40]
```

Modifying List Elements

```
numbers = [10, 20, 30, 40]
numbers[1] = 25
print(numbers)
```

List Operations

1 Concatenation (+)

```
a = [1, 2, 3]
b = [4, 5]
c = a + b
print(c)
```

2 Repetition (*)

```
pattern = [0, 1]
print(pattern * 3)
```

Output:

```
[0, 1, 0, 1, 0, 1]
```

Important List Methods

1. **`append()`**

```
nums = [1, 2, 3]
nums.append(4)
```

2. **`insert()`**

```
nums.insert(1, 10)
```

3. **`extend()`**

```
nums.extend([5, 6])
```

4. **`remove()` vs `pop()`**

Example 1: Average of Marks

```
marks=[78,85,90,66]
avg =sum(marks)/ len(marks)
print("Average:", avg)
```

Example 2: Filtering Values

```
temps=[32,35,30,38,40]
high = []

for t in temps:
    if t > 35:
        high.append(t)

print(high)
```

Example 3: Removing Missing Data

```
data=[10,None,20,None,30]
clean = []

for x in data:
    if x is not None:
        clean.append(x)

print(clean)
```

Introduction to Tuples

A **tuple** is:

- An **ordered** collection
- **Immutable** (cannot be changed)
- Allows **duplicate values**
- Faster and safer than lists

e.g t = (10, 20, 30)

Creating Tuples

```
t1 = (1, 2, 3)

t2 = ("Python", 3.10, True)
```

Single-Element Tuple

```
t = (5)    # this is int, not tuple

t = (5,)   # correct
```

Accessing Tuples

```
t = (10, 20, 30, 40)
```

Indexing

```
print(t[0])
print(t[-1])
```

Slicing

```
print(t[1:3])
print(t[:2])
```

Immutability Concept

Tuple values **cannot be modified** after creation.

```
t = (1, 2, 3)

t[0] = 10   # ERROR
```

Workaround

Convert tuple → list → tuple

```
t = (1, 2, 3)

lst = list(t)

lst[0] = 10

t = tuple(lst)

print(t)
```

Unpacking Tuples

Basic Unpacking

```
t = (10, 20, 30)
a, b, c = t
print(a, b, c)
```

Unpacking with *

```
t = (1, 2, 3, 4, 5)
a, b, *c = t
print(c)
```

Looping Through Tuples

Using `for` Loop

```
t = (10, 20, 30)
for x in t:
    print(x)
```

Using Index

```
for i in range(len(t)):
    print(t[i])
```

Joining Tuples

Using +

```
t1 = (1, 2)
t2 = (3, 4)
t3 = t1 + t2
```

Repetition

```
t = (1, 2)
print(t * 3)
```

Tuple Methods

```
t = (1, 2, 2, 3)
```

count()

```
print(t.count(2))
```

OUTPUT= 2

index()

```
print(t.index(3))
```

OUTPUT= 3

```
def stats(nums):
    #check type of list
    return min(nums), max(nums)

result = stats([10, 20, 30])
print(result) #check type of result
```

Introduction to Dictionaries

What is a Dictionary

A **dictionary** is:

- A collection of **key–value pairs**
- **Unordered** (conceptually)
- **Mutable**
- Keys are **unique**

```
student = {
    “
roll": 101,
    "name": "Anita",
    "marks": 89
}
```

Creating Dictionaries

Creating an Empty Dictionary

```
d1 = {}

d2 = dict()
```

Creating Dictionary with Values

```
emp = {
    "id": 1,
    "name": "Ravi",
    "salary": 45000
}
print(emp)
```

Using dict() Constructor

```
person = dict(name="Sita", age=22, city="Pune")

print(person)
```

Creating Dictionary from List of Tuples

```
data = [("a", 1), ("b", 2)]

d = dict(data)

print(d)
```

Accessing Dictionary Elements

```
emp = {
    "id": 1,
    "name": "Ravi",
    "salary": 45000
}
print(emp)

print(emp["name"])
```

Adding and Updating Dictionary Elements

Adding New Key–Value Pair

```
emp = {
    "id": 1,
    "name": "Ravi",
    "salary": 45000
}
print(emp)
```

```
emp["dept"] = "HR"
print(emp)
```

Updating Existing Value

```
emp["salary"] = 50000
print(emp)
```

Same syntax → add or update

`setdefault()` Method

Why `setdefault()`?

- Avoids `KeyError`
- Adds key **only if it doesn't exist**

```
student = {"name": "Anita", "marks": 85}

student.setdefault("age", 20)
print(student)
```

Output:

```
{'name': 'Anita', 'marks': 85, 'age': 20}
```

When Key Already Exists

```
student.setdefault("marks", 90)
print(student)
```

Output:

```
{'name': 'Anita', 'marks': 85, 'age': 20}
```

<u>Existing value is NOT overwritten.</u>

Counting Example

```
data = ["apple", "banana", "apple", "orange", "banana"]
count = {}

for item in data:
    count.setdefault(item, 0)
    count[item] += 1

print(count)
```

Output:

```
{'apple': 2, 'banana': 2, 'orange': 1}
```

Introduction to Sets

A **set** is:

- Unordered
- Mutable
- Does NOT allow duplicates
- Does NOT support indexing

Creating a Set

```
s = {1, 2, 3, 4}
print(s)
```

Output:

{1, 2, 3, 4}

Creating Empty Set

```
s = {}
print(type(s))
```

Output:

```
<class 'dict'>
```

Correct way:

```
s = set()
```

Removing Data from Sequences Using Sets

Remove Duplicates from List

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)
print(unique_numbers)
```

Output:

```
{1, 2, 3, 4, 5}
```

Removing Specific Elements

remove()

```
s = {1, 2, 3}

s.remove(2)

print(s)
```

If element not present → Error

Output:

```
{1, 3}
```

discard()

```
s = {1, 2, 3}
s.discard(5)
print(s)
```

Output:

```
{1, 2, 3}
```

No error. Safer option.

pop()

```
s = {10, 20, 30}

s.pop()

print(s)
```

Removes random element.

Performing Common Data Set Operations

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

Union (All elements)

```
print(A | B)
print(A.union(B))
```

Output:

```
{1, 2, 3, 4, 5, 6}
```

Intersection (Common elements)

```
print(A & B)
print(A.intersection(B))
```

Output:

```
{3, 4}
```

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

Difference (Only in A)

```
print(A - B)
```

Output:

```
{1, 2}
```

Symmetric Difference (Not common)

```
print(A ^ B)
```

Output:

```
{1, 2, 5, 6}
```

Subset and Superset

```
X = {1, 2}

Y = {1, 2, 3, 4}


print(X.issubset(Y))

print(Y.issuperset(X))
```

Output:

```
True

True
```

JSON in Python

What is JSON?

Definition

JSON = JavaScript Object Notation

It is a **lightweight data format** used to store and exchange data.

📌 It looks like a Python dictionary.

Function	Purpose
dumps()	Dict → JSON string
loads()	JSON string → Dict
dump()	Dict → JSON file
load()	JSON file → Dict

Working with JSON in Python

1.We use **json module** (standard library)

```
import json
```

2.**Convert Python Dictionary to JSON (dump)**

Example

```
import json
weather = {
    "day": 1,
    "temperature": 32,
    "rainfall": 5
}
json_data = json.dumps(weather)
print(json_data)
```

Output

```
{"day": 1, "temperature": 32, "rainfall": 5}
```

Convert JSON to Python Dictionary (loads)

```
import json

json_data = '{"day": 1, "temperature": 32}'

data = json.loads(json_data)

print(data)
print(type(data))
```

Output

```
{'day': 1, 'temperature': 32}
<class 'dict'>
```